

Java Web 实验报告

Session 会话管理与 Servlet 过滤器应用

实验主题：Session 访问统计、Session 购物车、编码过滤器、权限过滤器、注册验证过滤器、监听器在线统计

1 实验目的

通过本次实验掌握 HttpSession 的创建与属性存取、基于 Session 的购物车模拟、Servlet Filter 过滤器链的拦截与放行机制，以及 HttpSessionListener 监听会话生命周期实现在线用户统计的方法。

2 实验环境

操作系统	Windows 11
JDK	17
Web 容器	Apache Tomcat 10+ (Jakarta EE)
构建工具	Maven 3.x
技术规范	Jakarta Servlet 6.x + JSP 3.x

3 Session 访问次数统计

3.1 设计思路

每次用户访问页面时，Servlet 通过 request.getSession() 获取（或创建）当前浏览器对应的 Session 对象，从 Session 中读取名为 accesscount 的整数属性：若不存在则置为 1，否则自增 1 后写回。同一 Session 内反复刷新页面，计数持续累加；关闭浏览器或 Session 过期后计数重置。

3.2 核心代码

```
@WebServlet("/accessCount")
protected void doGet(...) {
    HttpSession session = request.getSession();
```

```

        Integer count = (Integer) session.getAttribute("accesscount");
        count = (count == null) ? 1 : count + 1;
        session.setAttribute("accesscount", count);
        // 输出当前访问次数与 Session ID
    }

```

3.3 文件结构

```

ch12-session-counter/
├── pom.xml
└── src/main/java/.../AccessCountServlet.java

```

3.4 运行截图

【图 3-1 多次刷新后访问次数递增的页面效果】

4 Session 模拟购物车

4.1 设计思路

购物车数据以 `Map<String, CartItem>` 形式存入 Session，键为商品 ID。用户点击「加入购物车」时，Servlet 从 Session 取出 Map，若商品已存在则数量 +1，否则新建 CartItem。模块提供三种提交策略对比：

- 直接 Forward：刷新会重复提交 POST，数量异常增加；
- Redirect（PRG 模式）：POST 后重定向 GET，刷新不会重复加购；
- Session 防重：用 requestId 集合记录已处理请求，拦截重复 POST。

4.2 核心代码

```

// CartSupport: Session 购物车读写
public static void addProduct(HttpSession session, String mode, Product p) {
    Map<String, CartItem> cart = getCart(session, mode);
    CartItem item = cart.get(p.getId());
    if (item == null) cart.put(p.getId(), new CartItem(p));
    else item.increment();
}

// RedirectCartServlet: PRG 防重复提交
CartSupport.addProduct(session, "redirect", product);
response.sendRedirect(contextPath + "/shop?tab=redirect");

```

4.3 文件结构

```

ch12-session-cart/
├── model/   CartItem.java, Product.java
└── service/ CartSupport.java, CatalogService.java

```

```
└─ *CartServlet.java, ShopServlet.java
└─ webapp/WEB-INF/shop.jsp
```

4.4 运行截图

【图 4-1 商城页面三种购物车模式对比】

【图 4-2 加入商品后 Session 中购物车列表】

5 过滤器解决中文乱码

5.1 设计思路

Tomcat 默认以 ISO-8859-1 解析 POST 请求体，直接读取中文参数会出现乱码。EncodingFilter 在请求到达 Servlet 之前统一设置 request/response 字符编码为 UTF-8，并指定 HTML 响应的 Content-Type，使注册表单提交的中文昵称能正确显示。

5.2 核心代码

```
public void doFilter(ServletRequest req, ServletResponse resp, FilterChain chain)
{
    req.setCharacterEncoding("UTF-8");
    resp.setCharacterEncoding("UTF-8");
    ((HttpServletResponse) resp).setContentType("text/html;charset=UTF-8");
    chain.doFilter(req, resp); // 放行到 RegServlet
}
```

web.xml 中将 EncodingFilter 映射到 /reg 路径，register.html 表单提交中文昵称即可验证效果。

5.3 文件结构

```
ch14-filter-intro/
└─ EncodingFilter.java
└─ RegServlet.java
└─ webapp/register.html
└─ WEB-INF/web.xml (filter-mapping → /reg)
```

5.4 运行截图

【图 5-1 提交中文昵称后注册结果页正确显示】

6 过滤器进行用户权限访问控制

6.1 设计思路

采用「登录校验 + 角色鉴权」两级过滤器链：

- LoginValidatorFilter：拦截 POST /login，比对 Session 中的验证码，错误则 forward 回登录页；
- LoginServlet：验证账号密码，将 username、role 写入 Session；
- PermissionFilter：拦截 /admin/* 与 /user/*，未登录重定向登录页，非 admin 角色访问管理页返回 403。

6.2 核心代码

```
// PermissionFilter
HttpSession session = httpReq.getSession(false);
if (session == null || session.getAttribute("username") == null) {
    httpResp.sendRedirect(contextPath + "/login.jsp?error=2");
    return;
}
if (uri.contains("/admin/") && !"admin".equals(session.getAttribute("role"))) {
    httpResp.sendError(403, "权限不足");
    return;
}
chain.doFilter(request, response);
```

6.3 文件结构

```
ch14-filter-intro/
├── demo/filter/LoginValidatorFilter.java
├── demo/filter/PermissionFilter.java
├── controller/LoginServlet.java
├── webapp/login.jsp, admin/admin.jsp, user/user.jsp
└── WEB-INF/web.xml
```

6.4 运行截图

【图 6-1 管理员登录后进入 admin 页面】

【图 6-2 普通用户访问 admin 返回 403】

7 过滤器验证用户注册

7.1 设计思路

ResValidatorFilter 在 RegisterServlet 处理之前对表单参数做服务端校验：用户名非空且长度 3~20 位、密码非空且长度 6~20 位。校验失败时将错误信息与已填字段 forward 回 register.jsp，避免无效数据进入业务层。

7.2 核心代码

```
public void doFilter(...) {
    String username = request.getParameter("username");
    String password = request.getParameter("password");
    if (isBlank(username)) errorMsg = "用户名不能为空! ";
    else if (username.length() < 3 || username.length() > 20)
        errorMsg = "用户名长度必须在 3~20 位之间! ";
    else if (password.length() < 6) errorMsg = "密码长度不足! ";

    if (errorMsg != null) {
        request.setAttribute("errorMsg", errorMsg);
        request.getRequestDispatcher("/register.jsp").forward(...);
        return;
    }
    chain.doFilter(request, response);
}
```

7.3 文件结构

```
ch14-filter-intro/
├── ResValidatorFilter.java
├── RegisterServlet.java
├── webapp/register.jsp
└── WEB-INF/web.xml (filter-mapping → /register)
```

7.4 运行截图

【图 7-1 用户名过短时的过滤器拦截提示】

【图 7-2 校验通过后 RegisterServlet 注册成功页】

8 监听器完成在线人数统计

8.1 设计思路

ch11-login-keeper 模块通过 HttpSessionListener 与 SessionManager 协作实现在线会话统计：

- 用户登录成功后，LoginServlet 调用 SessionManager.login(sessionId, username) 将 Session 登记到内存 Map；
- LoginObserver 监听 sessionDestroyed 事件，Session 过期或用户退出时自动从 Map 移除，防止脏数据；
- welcome.jsp 读取 activeSessions.size() 展示当前活跃会话数，即在线用户数。

8.2 核心代码

```
@WebListener
public class LoginObserver implements HttpSessionListener {
    public void sessionDestroyed(HttpSessionEvent se) {
        SessionManager.getInstance().logout(se.getSession().getId());
    }
}

// LoginServlet 登录成功时
sessionManager.login(session.getId(), username);

// welcome.jsp 展示
activeSessions.size() // 活跃会话 / 在线人数
```

8.3 文件结构

```
ch11-login-keeper/
├── LoginObserver.java      (HttpSessionListener)
├── SessionManager.java    (ConcurrentHashMap 维护在线 Session)
├── LoginServlet.java
└── webapp/welcome.jsp    (展示活跃会话统计)
```

8.4 运行截图

【图 8-1 welcome 控制面板显示活跃会话数量】

【图 8-2 多用户登录后会话列表】

9 实验总结

本次实验围绕 Session 与 Filter 两条主线展开：Session 负责在服务器端为每个浏览器维持独立状态，可用于访问计数、购物车等场景；Filter 作为请求链路上的横切组件，可集中处理编码、鉴权、表单校验等通用逻辑，避免在每个 Servlet 中重复编写。监听器则补充了 Session 生命周期的自动化管理，配合 SessionManager 实现可靠的在线用户统计。

实验中需注意：Filter 的 mapping 顺序决定拦截范围；POST 请求必须在读取参数前设置 UTF-8 编码；PRG 模式可有效避免购物车重复提交；权限控制应同时校验「是否登录」与「角色是否匹配」两层条件。